Experiment 10

Student Name: Krish Latawa
UID: 24BAI70541
Branch: CSE - AIML
Section/Group: 24AIT_KRG G1
Semester: 4
Date of Performance: 17/4/26
Subject Name: DBMS
Subject Code: 24CSH-298

Aim

To design a trigger that automatically implements the functionality of a primary key, ensuring unique identification of records without manual intervention.

Software Requirements

- Database Management System:
  - Oracle Database Express Edition (Oracle XE)
  - PostgreSQL Database
- Database Administration Tool / Client Tool:
  - Oracle SQL Developer (for Oracle XE)
  - pgAdmin (for PostgreSQL)

Objectives

- To create a database trigger that automatically enforces primary key constraints or generates unique key values, replicating the functionality of a stored procedure.

Problem Statement

In enterprise databases, primary keys must be unique for every record. Manual assignment of keys can lead to errors.
Design a trigger that:
Automatically generates or enforces a primary key value during record insertion.

Implements the logic similar to a stored procedure.
Demonstrates automated primary key functionality for a table.
(Industry relevance: Amazon, Flipkart, Oracle)

Theory

A trigger can be designed to automatically enforce the functionality of a primary key by ensuring that each record inserted into a table is uniquely identifiable without requiring manual checks. This is achieved by defining a trigger that activates before or after an insert operation and validates whether the incoming data violates uniqueness constraints. If a duplicate value is detected in the designated key column, the trigger can prevent the insertion or raise an error, thereby maintaining data integrity. Additionally, such a trigger can be used to generate unique values dynamically (for example, using sequences or custom logic) when a primary key is not explicitly provided. This approach helps in maintaining consistency, avoiding redundancy, and ensuring that each record can be uniquely referenced within the database system.

Practical/Experiment Steps

- Identify the table requiring automated primary key enforcement.
- Design a trigger that activates before insert operations.
- Ensure that every new record receives a unique primary key automatically.
- Validate the trigger by inserting multiple records and verifying unique keys.

Procedure

1. **Create the table**
   Define a table without explicitly enforcing a primary key constraint (or assume you want custom control).
2. **Identify the key column**
   Choose the column that should behave like a primary key (e.g., id).

3. **Write a BEFORE INSERT trigger**
   Create a trigger that fires before inserting a new record.
4. **Check for duplicate values**
   Inside the trigger, query the table to see if the incoming key value already exists.
5. **Handle duplicates**
   If a duplicate is found, raise an error and prevent insertion.
6. **(Optional) Auto-generate key**
   If no key is provided, assign a unique value automatically (e.g., using MAX(id)+1 or a sequence).
7. **Test the trigger**
   Insert records to verify:
   - Unique values are accepted
   - Duplicate values are rejected

Code

```
-- Step 1: Create table
        CREATE TABLE students (
           id INT,
           name VARCHAR(50)
        );

-- Step 2: Create trigger function
        CREATE OR REPLACE FUNCTION check_unique_id()
        RETURNS TRIGGER AS $$
        BEGIN

 -- Step 3: Auto-generate ID if NULL
           IF NEW.id IS NULL THEN
              SELECT COALESCE(MAX(id), 0) + 1 INTO NEW.id FROM
        students;
```

```
        END IF;

 -- Step 4: Check for duplicate ID
        IF EXISTS (SELECT 1 FROM students WHERE id = NEW.id) THEN
           RAISE EXCEPTION 'Duplicate ID not allowed!';
        END IF;

        RETURN NEW;
      END;
      $$ LANGUAGE plpgsql;

-- Step 5: Create trigger
      CREATE TRIGGER before_insert_students
      BEFORE INSERT ON students
      FOR EACH ROW
      EXECUTE FUNCTION check_unique_id();

-- Step 6: Testing
      INSERT INTO students VALUES (101, 'Krish');   -- Works
      INSERT INTO students VALUES (102, 'Rahul');   -- Error
      INSERT INTO students VALUES (NULL, 'Aman'); -- Auto-generated ID
```

Input/Output Analysis

SQL INPUT QUERIES:-

```
CREATE TABLE students (
id INT,
name VARCHAR(50)
);
```

OUTPUT:-

```
Data Output   Messages   Notifications
CREATE TABLE

Query returned successfully in 79 msec.
```

SQL INPUT QUERIES:-

```
-- Step 2: Create trigger function
        CREATE OR REPLACE FUNCTION check_unique_id()
        RETURNS TRIGGER AS $$
        BEGIN

 -- Step 3: Auto-generate ID if NULL
            IF NEW.id IS NULL THEN
                SELECT COALESCE(MAX(id), 0) + 1 INTO NEW.id FROM
        students;
            END IF;

 -- Step 4: Check for duplicate ID
            IF EXISTS (SELECT 1 FROM students WHERE id = NEW.id) THEN
                RAISE EXCEPTION 'Duplicate ID not allowed!';
            END IF;

            RETURN NEW;
        END;
        $$ LANGUAGE plpgsql;
```

OUTPUT:-

```
Data Output   Messages   Notifications
CREATE FUNCTION

Query returned successfully in 1 secs 333 msec.
```

SQL INPUT QUERIES:-

```
-- Step 5: Create trigger
CREATE TRIGGER before_insert_students
BEFORE INSERT ON students
FOR EACH ROW
EXECUTE FUNCTION check_unique_id();
```

OUTPUT:-

```
Data Output   Messages   Notifications

CREATE TRIGGER

Query returned successfully in 40 msec.
```

SQL INPUT QUERIES:-

```
-- Step 6: Testing
INSERT INTO students VALUES (101, 'Krish');   -- Works
INSERT INTO students VALUES (102, 'Rahul');   -- Error
INSERT INTO students VALUES (NULL, 'Aman'); -- Auto-generated ID
```

OUTPUT:-

```
Data Output   Messages   Notifications

INSERT 0 1

Query returned successfully in 71 msec.
```

Learning Outcomes

- After completing this experiment, the learner will be able to:
- Understand the purpose and working of database triggers.
- Implement automated primary key functionality using triggers.

- Ensure data integrity without manual key assignment.
- Apply trigger-based automation in real-world enterprise applications such as Amazon, Flipkart, and Oracle.